SOFTWARE REQUIREMENTS SPECIFICATION

for

CampusXchange

A Campus-Verified Marketplace for Selling, Lending and Exchange

Version 1.0

Prepared by (Group 3C-14, BE Third Year, CSE):

1. Garav Goyal (1024030098)
2. Dhruv Rajput (1024030102)
3. Parth Zutshi (1024030116)
4. Harjap Singh Benipal (1024030118)

Submitted to: Dr. Anushka
UCS503P – Software Engineering Laboratory

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

October 2026

Contents

1. Introduction **5**
- 1.1. Purpose . . . 5
- 1.2. Document Conventions . . . 5
- 1.3. Intended Audience and Reading Suggestions . . . 5
- 1.4. Product Scope . . . 5
- 1.5. Project Objectives . . . 6
- 1.6. Definitions, Acronyms and Abbreviations . . . 6
- 1.7. References . . . 7

2. Overall Description **8**
- 2.1. Product Perspective . . . 8
- 2.2. Product Functions . . . 8
- 2.3. Stakeholders . . . 9
- 2.4. User Classes and Characteristics . . . 10
- 2.5. Operating Environment . . . 10
- 2.6. Design and Implementation Constraints . . . 10
- 2.7. Assumptions and Dependencies . . . 11
- 2.8. Refinements to the Approved Proposal . . . 11

3. System Features and Functional Requirements **13**
- 3.1. Account, Sign-up and Verification . . . 13
- 3.2. Browsing and Search . . . 13
- 3.3. Listing Management . . . 14
- 3.4. Listing Moderation . . . 15
- 3.5. Chat and Negotiation . . . 15
- 3.6. Payment and Escrow . . . 16
- 3.7. QR-Verified Handoff . . . 16
- 3.8. Lend Return Cycle . . . 17
- 3.9. Disputes . . . 17
- 3.10. Notifications and Category Alerts . . . 18
- 3.11. Administration . . . 18

4. External Interface Requirements **19**
- 4.1. User Interfaces . . . 19
- 4.2. Hardware Interfaces . . . 19
- 4.3. Software Interfaces . . . 19
- 4.4. Communication Interfaces . . . 20

5. Non-Functional Requirements **21**
- 5.1. Performance . . . 21
- 5.2. Security . . . 21
- 5.3. Privacy . . . 22
- 5.4. Reliability and Availability . . . 22
- 5.5. Usability . . . 22
- 5.6. Portability and Compatibility . . . 22
- 5.7. Maintainability and Quality . . . 23

5.8. Scalability 23

6. System Models and Diagrams **24**
6.1. Use Case Diagram 24
6.2. Class / Entity-Relationship Diagram 24
6.3. Data Dictionary 25
6.4. State Models 26
6.5. Sequence Diagrams 26
6.6. Activity Diagrams 28

7. Business Rules **30**

8. Acceptance Criteria and Traceability **31**
8.1. Evaluation Metrics 31
8.2. Traceability Matrix 31

A. Known Gaps and Open Issues **33**

List of Figures

2.1. High-level system architecture (solid: implemented; dashed: planned) 8
2.2. End-to-end workflow of a CampusXchange transaction 9

6.1. Use case diagram 24
6.2. Class / ER diagram (13 tables) 25
6.3. Sequence diagram 1: sign-up, ID verification, listing and moderation 26
6.4. Sequence diagram 2: browsing, multi-buyer negotiation, payment and escrow . . . 27
6.5. Sequence diagram 3: QR handoff, lend return, settlement and disputes 27
6.6. Activity diagram 1: verification and moderation (with rejection loops) 28
6.7. Activity diagram 2: negotiation, payment, handoff and lend return 29
6.8. Activity diagram 3: dispute resolution and penalties 29

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of **CampusXchange**, a mobile marketplace in which verified students of Thapar Institute of Engineering and Technology can sell, lend and exchange everyday items such as textbooks, calculators, lab gear, cycles and electronics.

The document is the agreed reference for what the system must do. It is written to be consistent with the project proposal approved on 19 August 2026, and it records explicitly where the design has since been refined (Section 2.8). It covers the complete system, including the payment, handoff and dispute modules that are designed but not yet implemented at the time of writing.

1.2. Document Conventions

- Every requirement carries a unique identifier: `FR-<area>-<n>` for functional requirements, `NFR-<area>-<n>` for non-functional requirements and `BR-<n>` for business rules.
- **Priority** is one of *High* (essential to the core workflow; the system is not usable without it), *Medium* (important, but the core workflow still functions without it) or *Low* (desirable enhancement).
- **Status** records progress at the time of writing: *Implemented* (built and verified on device), *Partial* (built, but part of the requirement is not yet enforced) or *Planned* (fully designed and modelled; database support exists; implementation scheduled for the next phase).
- The words *shall* (mandatory), *should* (recommended) and *may* (optional) are used in their usual requirements sense.
- Database tables, columns and API routes appear in `monospace`.

1.3. Intended Audience and Reading Suggestions

- **Course instructor and evaluators** – Chapter 1 and 2 for scope and context; Chapter 3 and 5 for the requirements themselves.
- **Developers (the project team)** – Chapters 3, 4 and 6 give the exact behaviour, interfaces and data model to build against.
- **Testers** – every requirement in Chapters 3 and 5 is written to be verifiable; Chapter 8 lists the acceptance criteria.
- **Pilot administrators / moderators** – Sections 3.4, 3.9 and 3.11 describe their responsibilities in the system.

1.4. Product Scope

Campus commerce today runs on hostel WhatsApp groups, batch chats and noticeboards that were never designed for hundreds of people who mostly do not know each other to trade safely. CampusXchange replaces these with a single, college-only app in which:

- only holders of a valid `@thapar.edu` address can register, and each account also carries a roll number and an administrator-reviewed student ID card;
- items can be listed for **Sell**, **Lend/Rent** or **Exchange (barter)** from one unified, searchable feed;
- no listing is published until an administrator has reviewed it;
- buyers and sellers negotiate through in-app chat and structured offers that the system records;
- payment is held in escrow and released only after a QR-verified, in-person handoff on campus, with condition photographs as evidence;
- disputes are adjudicated by an administrator using that evidence.

Out of scope for this version: shipping or courier delivery, cross-campus listings, a custom-trained machine-learning model, OCR or third-party KYC verification of ID documents, and a separate web application for students.

1.5. Project Objectives

O1. Verified access: restrict participation to genuine Thapar students, enforced in the database rather than only in the client.

O2. Unified listing surface: one searchable, filterable feed covering Sell, Lend and Exchange.

O3. Moderated publication: no listing reaches the public feed without passing through human review.

O4. Recorded negotiation: price agreement is captured as structured offers rather than free-text chat.

O5. Safe settlement: escrow-held payment released only after a QR-verified handoff, with support for the lend return cycle.

O6. Recourse: an evidence-based dispute process run by administrators.

O7. Measurable outcome: median time-to-listing-clearance (TLC) of at most 4 hours during the pilot.

1.6. Definitions, Acronyms and Abbreviations

Term	Meaning
API	Application Programming Interface (here, the NestJS REST backend)
Admin	A user whose `users.role` is `admin`; performs moderation, ID verification and dispute resolution
Escrow	Holding a buyer's payment until the handoff is confirmed
Guest	A person using the app without signing in; may browse only
Handoff	The physical, on-campus meeting at which an item changes hands
Listing	An item offered for Sell, Lend or Exchange
OTP	One-Time Password, a 6-digit code sent to the student's college email
QR token	A signed, single-use, time-bound code that identifies one transaction at handoff

Term	Meaning
RLS	Row-Level Security (PostgreSQL feature)
Signed URL	A time-limited link to a private file in object storage
Student	A signed-up user whose ID card is not yet verified
Verified Student	A student whose `id_verified` is `verified`
TLC	Time to Listing Clearance: time from listing submission to moderation decision
UML	Unified Modeling Language

1.7. References

1. CampusXchange Project Proposal, UCS503P, submitted 19 August 2026.
2. CampusXchange Mid-Semester Evaluation Report (`report/main.tex`).
3. CampusXchange API Contract (`docs/api-contract.md`).
4. CampusXchange database schema (`supabase/migrations/`).
5. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

2. Overall Description

2.1. Product Perspective

CampusXchange is a new, self-contained product that replaces informal group-chat trading. It follows a three-tier architecture (Figure 2.1):

- **Mobile client** – a React Native (Expo) app; the only user-facing surface. Students, guests and administrators all use it; administrators see an extra *Review* tab.
- **Backend API** – a stateless NestJS service organised into feature modules (profile, listings, chat, admin) behind a guard layer that authenticates every request.
- **Data platform** – Supabase, providing PostgreSQL, the authentication service (email OTP) and object storage (a private `id-cards` bucket and a public `listing-images` bucket).

Authentication traffic goes directly from the app to the identity provider; all application data passes through the backend.

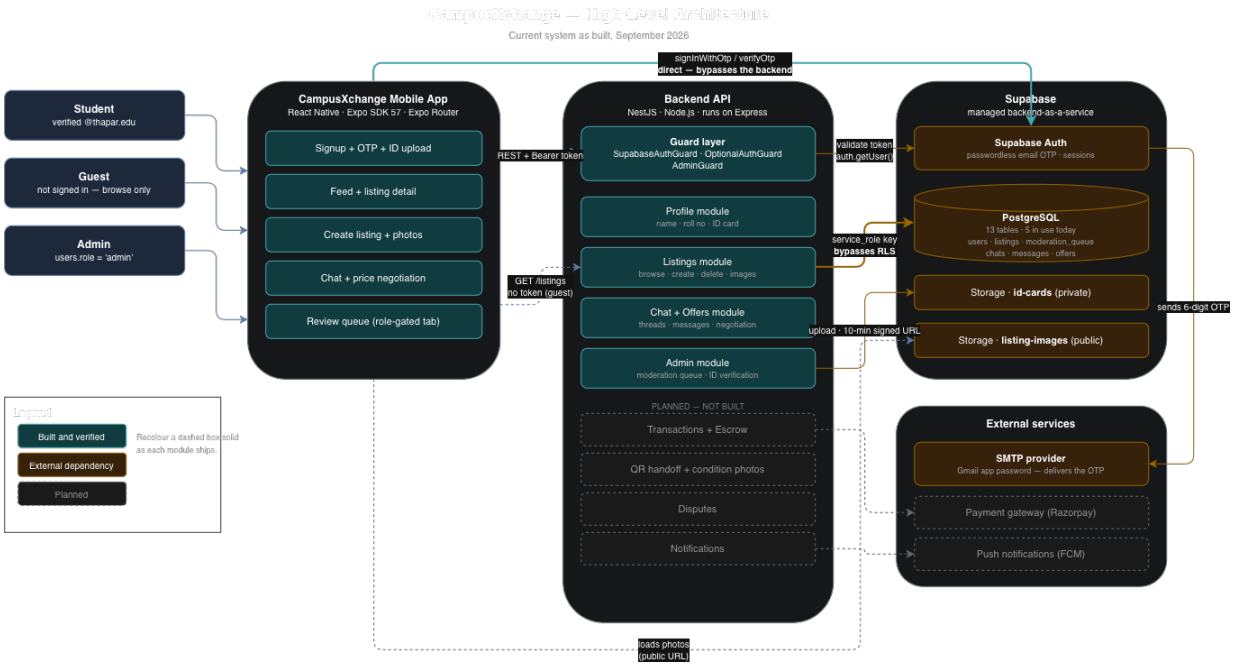


Figure 2.1.: High-level system architecture (solid: implemented; dashed: planned)

2.2. Product Functions

At a high level the system shall allow users to:

1. sign up and log in with a college email and OTP, provide a roll number and upload a student ID card for verification;
2. browse and search live listings by type and category, including as a guest;
3. create, view and delete listings with photographs, condition and a campus meeting point;

4. have every new listing reviewed by an administrator before it goes live;
5. chat with the other party and negotiate a price through offers;
6. pay into escrow, receive a QR code, and complete a verified handoff with condition photographs;
7. complete a return leg for lend transactions;
8. raise and resolve disputes;
9. receive notifications and category-based alerts.

Figure 2.2 summarises the end-to-end workflow.

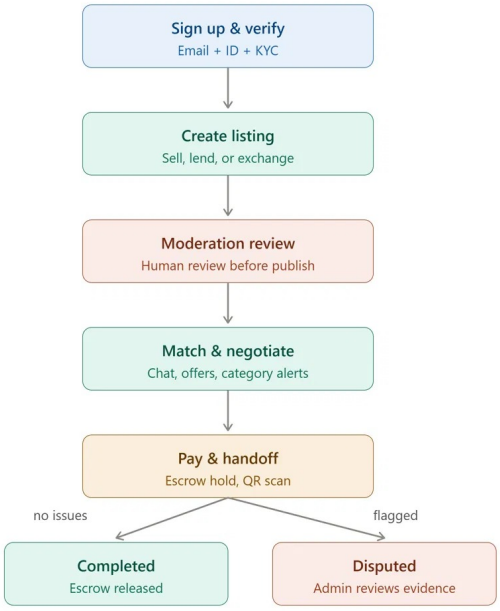


Figure 2.2.: End-to-end workflow of a CampusXchange transaction

2.3. Stakeholders

Table 2.1.: Stakeholders and their interests

Stakeholder	**Interest in the system**
Students (buyers, borrowers)	Find items cheaply and safely from people on their own campus
Students (sellers, lenders)	Reach the whole campus, get paid reliably, get borrowed items back
Administrators / moderators	Keep the marketplace free of unsafe or fake content; resolve disputes fairly with evidence
Thapar Institute	A safe, college-only trading space; protection of student identity data
Course instructor (Dr. Anushka)	A well-engineered, specified, tested and measurable project
Development team	Clear, realistic requirements to build and test against

Table 2.1.: Stakeholders and their interests

Stakeholder	Interest in the system
External service providers	Supabase (auth, database, storage), payment gateway and push-notification service, whose limits constrain the design

2.4. User Classes and Characteristics

Table 2.2.: User classes

User class	Description and permissions
Guest	Not signed in. May browse and search live listings and view listing details. The meeting location is hidden. Cannot chat, list, or transact.
Student	Signed in with a verified `@thapar.edu` email; ID card not yet approved. May browse, upload an ID card, set a roll number and receive alerts.
Verified Student	ID card approved by an admin. Inherits all Student rights and may additionally create listings, chat and negotiate, pay, complete handoffs and returns, and raise disputes.
Administrator	A student account with `role = admin`. Reviews ID cards, moderates listings, resolves disputes, applies penalties and views platform analytics.
Notification Service (external)	Secondary actor; delivers push/email notifications. Never initiates a use case.
Payment Gateway (external)	Secondary actor; collects payment and supports escrow hold, release and refund.

Users are expected to be comfortable with smartphone apps, but may use budget Android devices and unreliable hostel Wi-Fi. Administrators are trained student volunteers or staff (2–3 in the pilot).

2.5. Operating Environment

- **Client:** Android and iOS smartphones running the Expo (React Native, Expo SDK 57) app; during development, run through Expo Go.
- **Backend:** Node.js (v22) running NestJS; currently run locally, to be deployed on a cloud platform for the pilot.
- **Data platform:** Supabase-hosted PostgreSQL, Supabase Auth and Supabase Storage.
- **Network:** HTTPS over campus Wi-Fi or mobile data.

2.6. Design and Implementation Constraints

C1. Sign-up is restricted to the `@thapar.edu` domain; single-campus pilot with no cross-campus visibility.

C2. Technology stack is fixed: React Native (Expo) + TypeScript for the client, NestJS + TypeScript for the backend, Supabase (PostgreSQL) for data, authentication and storage.

C3. Passwords are not stored by the application; authentication is passwordless (email OTP) and handled entirely by Supabase Auth.

C4. The Supabase `service_role` key may exist only on the backend and must never be shipped inside the mobile app.

C5. Moderation and matching must be rule- or API-assisted only – no custom-trained ML model. Humans make every final decision; nothing is auto-rejected or auto-banned.

C6. No OCR, barcode scanning or third-party KYC vendor is used for ID verification.

C7. All handoffs are physical and on campus at pre-seeded meeting points; no shipping or courier integration.

C8. The app must remain usable on budget Android phones and patchy networks.

C9. Development is by a four-person student team within one semester, using free tiers of hosted services (e.g. Supabase's built-in mailer is rate-limited, so custom SMTP is required).

C10. Every push must pass CI (mobile type-check and backend build on GitHub Actions).

2.7. Assumptions and Dependencies

- Every student has access to their `@thapar.edu` mailbox to receive OTPs.
- Students possess a physical college ID card they can photograph.
- Supabase services (Auth, PostgreSQL, Storage) remain available within free-tier limits during the pilot.
- A payment gateway with escrow or split-payment support (e.g. a sandbox/test mode) is available for the payment phase.
- An SMTP provider is configured so that OTP emails contain the 6-digit code rather than a magic link.
- 2–3 volunteer administrators are available to keep TLC within target.

2.8. Refinements to the Approved Proposal

The following changes were made deliberately after the proposal was approved. They are recorded here so that this SRS remains traceable to it.

Table 2.3.: Documented deviations from the approved proposal

Area	**Proposal said**	**This SRS specifies, and why**
Identity verification	Roll number only, entered by hand; "no ID card image or scan"	Roll number *and* a photograph of the student ID card, reviewed manually by an admin. Format-checking alone could not prove a roll number belongs to the person using it. Privacy is protected by a private bucket, short-lived signed URLs for admins only, and per-user storage paths (NFR-SEC-4, NFR-PRV-1).

Table 2.3.: Documented deviations from the approved proposal

Area	**Proposal said**	**This SRS specifies, and why**
Meeting location	Suggestions pre-seeded with campus landmarks	Implemented as a fixed dropdown of hostel blocks, academic blocks and landmarks, with no GPS, so no location permission is ever needed.
Guest browsing	Not mentioned	Guests may browse the feed; the meeting location is withheld server-side.
Admin interface	Separate web dashboard	An admin-only *Review* tab in the mobile app for the pilot; a web dashboard remains optional (FR-ADM-6).
Real-time chat	Real-time messaging	Chat currently polls every 4 seconds; Socket.io is planned (NFR-PRF-5). The API shape does not change.
AI moderation flagging	Off-the-shelf moderation API assigns a flag score	Schema column exists (`ai_flag_score`); not wired for the pilot. Every listing goes to human review, which satisfies the proposal's "mandatory human review".

3. System Features and Functional Requirements

Each feature below gives a short description, its priority, the main stimulus/response sequence, and the functional requirements.

3.1. Account, Sign-up and Verification

Description and priority: Gives students a passwordless account tied to their college email and verifies that they are real, current students. *Priority: High.*
Stimulus/response: Student enters email → system sends a 6-digit OTP → student enters OTP → account and profile are created → student enters roll number and uploads ID card → profile is marked *pending* → admin approves or rejects → student becomes *verified* or is asked to re-upload.

ID	Requirement	Priority	Status
FR-AUTH-1	The system shall let a user sign up and sign in using only an email address and a 6-digit OTP sent to that address.	High	Implemented
FR-AUTH-2	The system shall reject any email address not ending in `@thapar.edu` (case-insensitive) both in the app and in the database, so that no account is created and no OTP is sent.	High	Implemented
FR-AUTH-3	On creation of an authentication account, the system shall automatically create a matching profile row in `users` with role `student` and `id_verified = pending`.	High	Implemented
FR-AUTH-4	The system shall require the student to enter a roll number consisting of 9–10 digits, and shall store it on the profile.	High	Implemented
FR-AUTH-5	The system shall let the student upload a photograph of their student ID card (JPEG, PNG, HEIC or WebP; max 8 MB), store it privately, and set `id_verified` to `pending`.	High	Implemented
FR-AUTH-6	The system shall allow a student whose ID card was rejected to upload a new photograph.	Medium	Implemented
FR-AUTH-7	The system shall display the user's profile, including email, roll number, verification status and role.	Medium	Implemented
FR-AUTH-8	The system shall let the user sign out, ending the session on the device.	High	Implemented
FR-AUTH-9	The system shall keep the user signed in across app restarts until they sign out or the session expires.	Medium	Implemented
FR-AUTH-10	Where a registrar-provided student list is available, the system should cross-check the entered roll number against it.	Low	Planned

3.2. Browsing and Search

Description and priority: A single feed of live listings across all three transaction types. *Priority: High.*

ID	Requirement	Priority	Status
FR-BRW-1	The system shall display a feed of listings whose status is `live`, newest first, to both guests and signed-in users.	High	Implemented
FR-BRW-2	The system shall let users filter the feed by type (Sell, Lend, Exchange) and by category (Textbooks, Electronics, Lab & Drafter, Cycle & Commute, Sports Gear, Stationery).	High	Implemented
FR-BRW-3	The system shall let users search listings by keyword, case-insensitively, over title and description.	High	Implemented
FR-BRW-4	The system shall show a listing detail screen with photographs, title, description, category, condition, price or rent terms or wanted items, and meeting location.	High	Implemented
FR-BRW-5	For a request without a valid session, the backend shall replace the meeting location with `null` and mark it hidden; the app shall prompt the guest to sign in to see it.	High	Implemented
FR-BRW-6	The system shall prompt a guest to sign in when they attempt any action other than browsing (chat, list, offer).	High	Implemented
FR-BRW-7	The system shall return at most 100 listings per feed request; pagination should be added for scale.	Medium	Partial
FR-BRW-8	The system should allow sorting listings by proximity of the meeting point to the viewer's hostel.	Low	Planned

3.3. Listing Management

Description and priority: Verified students post items for Sell, Lend or Exchange. *Priority: High.*

ID	Requirement	Priority	Status
FR-LST-1	The system shall allow a verified student to create a listing with: type (sell / lend / exchange), title (3–120 characters), optional description (up to 2000 characters), category, condition (Like New / Good / Fair) and meeting location.	High	Partial[a]
FR-LST-2	The system shall require a price for a Sell listing, a rent amount (with unit day or week) for a Lend listing, and at least one wanted-item tag (max 10) for an Exchange listing, and shall reject a listing missing its type-specific field.	High	Implemented
FR-LST-3	The system shall let the user attach up to 6 photographs (images only, max 8 MB each) to a listing, stored in public storage under the owner's folder.	High	Implemented
FR-LST-4	The system shall offer the meeting location as a choice from pre-seeded campus points (Library Foyer, Hostel Blocks A–D, Academic Blocks A–B, Sports Complex, Main Gate, Cafeteria).	Medium	Implemented
FR-LST-5	The system shall save every new listing with status `pending_review` and never publish it directly.	High	Implemented
FR-LST-6	The system shall show a user all of their own listings at every status (pending review, live, reserved, completed, removed).	High	Implemented

ID	Requirement	Priority	Status
FR-LST-7	The system shall allow only the owner of a listing to delete it; any other user shall receive a "forbidden" response.	High	Implemented
FR-LST-8	The system should allow the owner to edit a listing; an edited live listing shall return to moderation.	Medium	Planned
FR-LST-9	The system shall mark a listing `reserved` once a buyer is selected and `completed` once the transaction closes.	High	Planned

[a] Listing creation works for every signed-in student; the check that the student is *verified* is not yet enforced on `POST /listings` (see Appendix A).

3.4. Listing Moderation

Description and priority: Every listing is reviewed by a human before it becomes visible. *Priority: High.*

ID	Requirement	Priority	Status
FR-MOD-1	When a listing is created, the system shall insert a corresponding entry into the moderation queue with status `pending`; if this fails, the listing shall be rolled back.	High	Implemented
FR-MOD-2	The system shall show administrators the queue of pending listings with full listing details.	High	Implemented
FR-MOD-3	An administrator shall be able to approve a listing, which sets the listing to `live` and records the reviewer and time.	High	Implemented
FR-MOD-4	An administrator shall be able to reject a listing with a reason, which sets the listing to `removed`.	High	Implemented
FR-MOD-5	The system shall record submission and decision timestamps so that TLC can be measured.	High	Implemented
FR-MOD-6	The system should notify the owner when their listing is approved or rejected, including the reason.	Medium	Planned
FR-MOD-7	The system may obtain a confidence score from an off-the-shelf moderation API to help prioritise the queue; it shall never auto-reject on that score.	Low	Planned

3.5. Chat and Negotiation

Description and priority: Private conversation between an interested buyer and the owner, with structured price offers. *Priority: High.*

ID	Requirement	Priority	Status
FR-CHT-1	The system shall let a signed-in user start a chat on another user's live listing; there shall be at most one thread per buyer per listing, and starting it again shall return the existing thread.	High	Implemented
FR-CHT-2	The system shall prevent a user from starting a chat on their own listing.	High	Implemented

ID	Requirement	Priority	Status
FR-CHT-3	The system shall let participants send text messages and view the thread's history in time order (up to the latest 200 messages).	High	Implemented
FR-CHT-4	The system shall list a user's chat threads, each with its listing and last message.	High	Implemented
FR-CHT-5	The system shall return "forbidden" to any user who is not a participant of a thread for every message or offer operation.	High	Implemented
FR-CHT-6	Either participant shall be able to propose a price offer; only the other participant shall be able to accept or reject it.	High	Implemented
FR-CHT-7	Accepting an offer shall automatically reject all other pending offers in that thread, and no further offers shall be allowed in that thread.	High	Implemented
FR-CHT-8	The app shall show messages and offers in one combined timeline ordered by time.	Medium	Implemented
FR-CHT-9	The seller shall be able to choose one buyer among several concurrent negotiations and finalise the deal, creating a transaction.	High	Planned
FR-CHT-10	Messages should be delivered in real time (WebSocket) rather than by polling.	Medium	Planned

3.6. Payment and Escrow

Description and priority: A shared transaction core used by all three modules. *Priority: High (next phase).*

ID	Requirement	Priority	Status
FR-PAY-1	When a deal is finalised, the system shall create a transaction recording listing, type, buyer, seller, agreed price and meeting location, with status `pending_payment`.	High	Planned
FR-PAY-2	The system shall collect the buyer's payment through the payment gateway and hold it in escrow (status `escrow_held`).	High	Planned
FR-PAY-3	For Lend transactions, the system shall record start date, end date, deposit amount and rent per day.	High	Planned
FR-PAY-4	The system shall release escrow to the seller only after a successful handoff (and, for Lend, a completed return).	High	Planned
FR-PAY-5	The system shall refund the buyer if the transaction is cancelled before handoff or a dispute is decided in the buyer's favour.	High	Planned
FR-PAY-6	The system shall show both parties the current transaction status.	Medium	Planned

3.7. QR-Verified Handoff

Description and priority: Confirms the physical meeting before money moves. *Priority: High (next phase).*

ID	Requirement	Priority	Status
FR-HND-1	After payment is held, the system shall issue a signed, single-use, time-bound QR token for the transaction (status `qr_issued`).	High	Planned
FR-HND-2	At the meeting, the counter-party shall scan the QR code in the app; the system shall accept it only if it is valid, unexpired, unused and belongs to that transaction.	High	Planned
FR-HND-3	The system shall require condition photographs to be captured at handoff and store them against the transaction.	High	Planned
FR-HND-4	On a valid scan, the system shall mark the token used, and for Sell/Exchange shall complete the transaction and release escrow.	High	Planned
FR-HND-5	Either party shall be able to flag a discrepancy at handoff instead of confirming, which moves the transaction to `disputed`.	High	Planned

3.8. Lend Return Cycle

Description and priority: Gives lending a memory: who borrowed what and when it is due back. *Priority: Medium.*

ID	Requirement	Priority	Status
FR-LND-1	For a Lend transaction, the system shall keep it open after handoff until the item is returned.	High	Planned
FR-LND-2	The system shall require return-stage condition photographs and a QR-verified return.	High	Planned
FR-LND-3	The system should remind the borrower before the due date and flag overdue items.	Medium	Planned
FR-LND-4	On a confirmed return with no reported issue, the system shall settle rent and refund the deposit.	High	Planned

3.9. Disputes

Description and priority: Recourse when something goes wrong. *Priority: Medium.*

ID	Requirement	Priority	Status
FR-DSP-1	A party to a transaction shall be able to raise one dispute per transaction with a reason.	High	Planned
FR-DSP-2	The system shall freeze escrow while a dispute is open.	High	Planned
FR-DSP-3	Administrators shall see open disputes with handoff and return condition photographs side by side.	High	Planned
FR-DSP-4	An administrator shall be able to request more evidence, then resolve the dispute with a written resolution, releasing or refunding escrow.	High	Planned
FR-DSP-5	An administrator may apply a penalty amount to the party at fault, per pre-defined penalty tiers.	Medium	Planned

ID	Requirement	Priority	Status
FR-DSP-6	The system shall notify both parties of each change in dispute status.	Medium	Planned

3.10. Notifications and Category Alerts

Priority: Medium.

ID	Requirement	Priority	Status
FR-NTF-1	The system shall store in-app notifications per user with a type, payload and read/unread flag.	Medium	Planned
FR-NTF-2	The system shall notify users of listing decisions, new messages and offers, transaction updates and dispute updates.	Medium	Planned
FR-NTF-3	The system should let a student subscribe to categories and alert them when a matching listing goes live (and match Exchange want-tags).	Low	Planned

3.11. Administration

Priority: High.

ID	Requirement	Priority	Status
FR-ADM-1	The system shall show an admin-only *Review* tab to users with `role = admin`.	High	Implemented
FR-ADM-2	The backend shall re-check the caller's admin role from the database on every `/admin` request, returning 403 to non-admins, regardless of what the app displays.	High	Implemented
FR-ADM-3	The system shall show admins pending ID cards, each via a signed URL valid for 10 minutes.	High	Implemented
FR-ADM-4	An admin shall be able to mark an ID card `verified` or `rejected`.	High	Implemented
FR-ADM-5	The system should show platform analytics: TLC (median), listing volume by type, transaction completion rate, dispute rate and on-time return rate.	Medium	Planned
FR-ADM-6	A separate web dashboard may be provided for administrators.	Low	Planned

4. External Interface Requirements

4.1. User Interfaces

The mobile app uses bottom-tab navigation. Screens:

Table 4.1.: Main screens

Screen	Purpose
Sign up / Sign in	Enter college email, then the 6-digit OTP
Onboarding: ID card	Enter roll number and photograph/upload the ID card
Home (feed)	Sell / Lend / Barter segmented control, category chips, search box, listing cards
Listing detail	Photos, details, meeting point (hidden for guests), "Chat with seller"
Create	Listing form with type-specific fields, photo picker, condition and meeting-point pickers
Chats / Chat thread	Thread list; messages and offer cards (accept / reject / counter) in one timeline
History (My listings)	Own listings with status badges; delete action
Profile	Account details, verification status, sign out
Review (admin only)	Moderation queue and ID-verification queue
Transaction, QR and Disputes (planned)	Payment, QR display/scan, condition-photo capture, dispute form

UI requirements: consistent colour and component library (buttons, chips, banners, empty states); every screen shall show a loading state, an empty state and a readable error message; pull-to-refresh on lists.

4.2. Hardware Interfaces

The app uses the phone's camera and photo library (ID card, listing photos, condition photos) and, in the next phase, the camera as a QR scanner. No GPS is required.

4.3. Software Interfaces

System	Interface
Supabase Auth	Called directly from the app: `signInWithOtp`, `verifyOtp`, `signOut`. Backend validates the bearer token with `auth.getUser`.
Supabase PostgreSQL	Accessed only by the backend using the service-role key; 13 tables (Section 6.3).

System	Interface
Supabase Storage	`id-cards` (private) and `listing-images` (public) buckets.
SMTP provider	Delivers OTP emails using custom templates showing the code.
Payment gateway (planned)	Create payment, hold, release and refund; sandbox mode for the pilot.
Push service (planned)	Expo push notifications for alerts.

4.4. Communication Interfaces

All client–server communication uses HTTPS and JSON (multipart form data for uploads). Authenticated requests carry `Authorization: Bearer <access token>`. Table 4.3 summarises the REST API.

Table 4.3.: REST API summary

Method	Route	Access	Purpose
GET	`/health`	Public	Liveness check
GET	`/profile/me`	Auth	Caller's profile
PATCH	`/profile/roll-number`	Auth	Set roll number
POST	`/profile/id-card`	Auth	Upload ID card
GET	`/listings`	Public	Live feed (filters: type, category, search)
GET	`/listings/:id`	Public	Listing detail
GET	`/listings/mine`	Auth	Own listings, all statuses
POST	`/listings`	Auth	Create listing (pending review)
POST	`/listings/images`	Auth	Upload a listing photo
DELETE	`/listings/:id`	Owner	Delete own listing
GET/POST	`/chats`	Auth	List / open a thread
GET/POST	`/chats/:id/messages`	Participant	Read / send messages
GET/POST	`/chats/:id/offers`	Participant	Read / make offers
POST	`/chats/offers/:id/accept|reject`	Recipient	Respond to offer
GET	`/admin/moderation`	Admin	Pending listings
POST	`/admin/moderation/:id/approve|reject`	Admin	Moderation decision
GET	`/admin/id-verifications`	Admin	Pending ID cards
POST	`/admin/id-verifications/:userId`	Admin	ID decision

5. Non-Functional Requirements

5.1. Performance

ID	Requirement	Priority	Status
NFR-PRF-1	The feed and listing-detail endpoints should respond within 1.5 s (95th percentile) over campus Wi-Fi under pilot load.	High	Planned
NFR-PRF-2	The system should support at least 100 concurrent users and semester-start spikes of several hundred new listings per day without degradation.	Medium	Planned
NFR-PRF-3	Feed queries shall use indexes on (`status`, `category`) and owner.	Medium	Implemented
NFR-PRF-4	Images shall be limited to 8 MB on upload, and the app should compress photos before uploading to save mobile data.	Medium	Partial
NFR-PRF-5	A sent chat message should appear for the other participant within 5 s (currently 4 s polling; real-time after WebSocket upgrade).	Medium	Implemented

5.2. Security

ID	Requirement	Priority	Status
NFR-SEC-1	Every non-public endpoint shall validate the bearer token before running any business logic and return 401 if it is missing, invalid or expired.	High	Implemented
NFR-SEC-2	Authorisation (admin role, listing ownership, chat participation, offer recipient) shall be enforced on the server; client-side checks are for display only.	High	Implemented
NFR-SEC-3	The service-role key shall exist only in the backend environment; secrets shall be kept in git-ignored `.env` files.	High	Implemented
NFR-SEC-4	ID card images shall be stored in a private bucket and exposed only to admins through signed URLs that expire in 10 minutes.	High	Implemented
NFR-SEC-5	Row-Level Security shall be enabled on all tables before any wider release, so the public (anon) key cannot read or write tables directly.	High	Planned
NFR-SEC-6	All input shall be validated at the API boundary (types, lengths, enums, file type and size).	High	Implemented
NFR-SEC-7	QR tokens shall be cryptographically signed, single-use and expire after a short window.	High	Planned
NFR-SEC-8	All traffic in deployment shall use HTTPS.	High	Planned

5.3. Privacy

ID	Requirement	Priority	Status
NFR-PRV-1	The system shall collect only the identity data needed: email, name, roll number and ID-card image.	High	Implemented
NFR-PRV-2	A guest shall never receive a listing's meeting location.	High	Implemented
NFR-PRV-3	A user's email and roll number shall not be shown to other students.	High	Implemented
NFR-PRV-4	Condition photographs shall be stored privately and visible only to the two parties and admins.	Medium	Planned

5.4. Reliability and Availability

ID	Requirement	Priority	Status
NFR-REL-1	Login, listing and payment shall be available at least 99% of the time during peak hours of the pilot.	High	Planned
NFR-REL-2	Related writes shall be atomic: a listing and its moderation entry are created together or not at all; escrow and transaction status shall never disagree.	High	Partial
NFR-REL-3	Accepting an offer shall leave a thread with exactly one accepted offer.	High	Implemented
NFR-REL-4	The app shall show clear, retryable errors when the network fails, rather than crashing.	Medium	Implemented

5.5. Usability

ID	Requirement	Priority	Status
NFR-USE-1	A new student should be able to sign up and post a first listing in under 5 minutes without help.	Medium	Planned
NFR-USE-2	Transaction status shall always be visible and in plain language; target average clarity rating ≥ 4 out of 5.	Medium	Planned
NFR-USE-3	Text shall be legible and touch targets adequately sized on small Android screens.	Medium	Implemented

5.6. Portability and Compatibility

ID	Requirement	Priority	Status
NFR-POR-1	The app shall run on both Android and iOS from one codebase.	High	Implemented
NFR-POR-2	The app should run acceptably on budget Android devices (2–3 GB RAM).	Medium	Planned

ID	Requirement	Priority	Status
NFR-POR-3	The backend shall be stateless so it can be deployed on any Node.js cloud host or container platform.	Medium	Implemented

5.7. Maintainability and Quality

ID	Requirement	Priority	Status
NFR-MNT-1	Code shall be written in TypeScript (strict mode) and organised by feature module in both app and backend.	High	Implemented
NFR-MNT-2	Each push shall trigger CI that type-checks the mobile app and builds the backend; failing builds shall not be merged.	High	Implemented
NFR-MNT-3	Schema changes shall be made through numbered migration files in `supabase/migrations`.	High	Implemented
NFR-MNT-4	Every API endpoint shall be documented in `docs/api-contract.md`.	Medium	Implemented
NFR-MNT-5	Unit and integration tests should cover the offer rules, moderation flow and transaction state transitions.	Medium	Planned

5.8. Scalability

ID	Requirement	Priority	Status
NFR-SCL-1	Listing and search queries should support pagination and indexing so they keep within NFR-PRF-1 as listings grow.	Medium	Partial
NFR-SCL-2	The system should be deployable with a managed database, a platform-hosted backend and a CI/CD pipeline to staging.	Medium	Planned

6. System Models and Diagrams

6.1. Use Case Diagram

Figure 6.1 shows four actors (Student, Verified Student, Administrator and the external Notification Service) and nineteen use cases, partitioned by privilege. Verified Student generalises Student, so verification acts as a gate: only verified students may list, negotiate, pay or raise disputes.

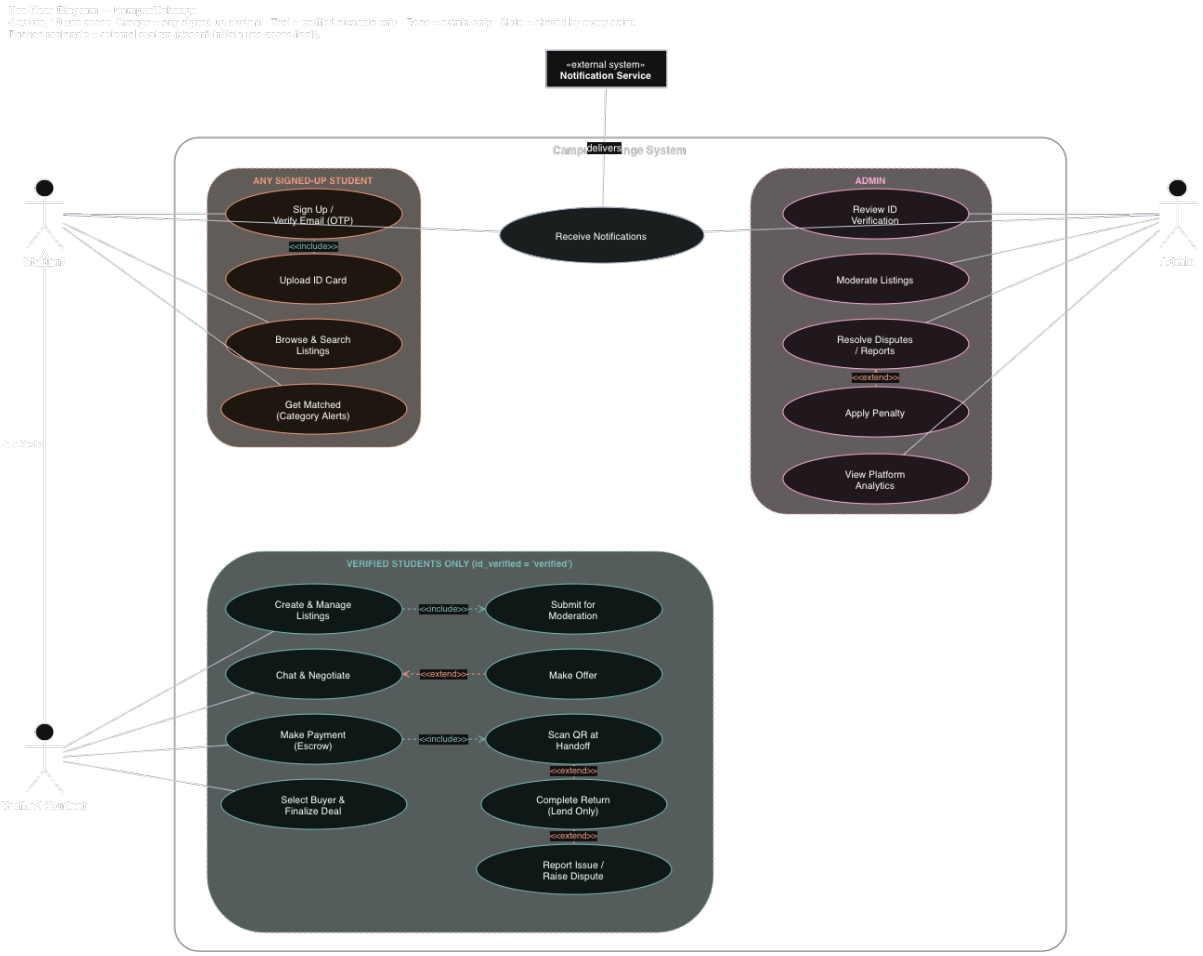


Figure 6.1.: Use case diagram

6.2. Class / Entity-Relationship Diagram

Figure 6.2 shows the thirteen database tables with their keys and constraints.

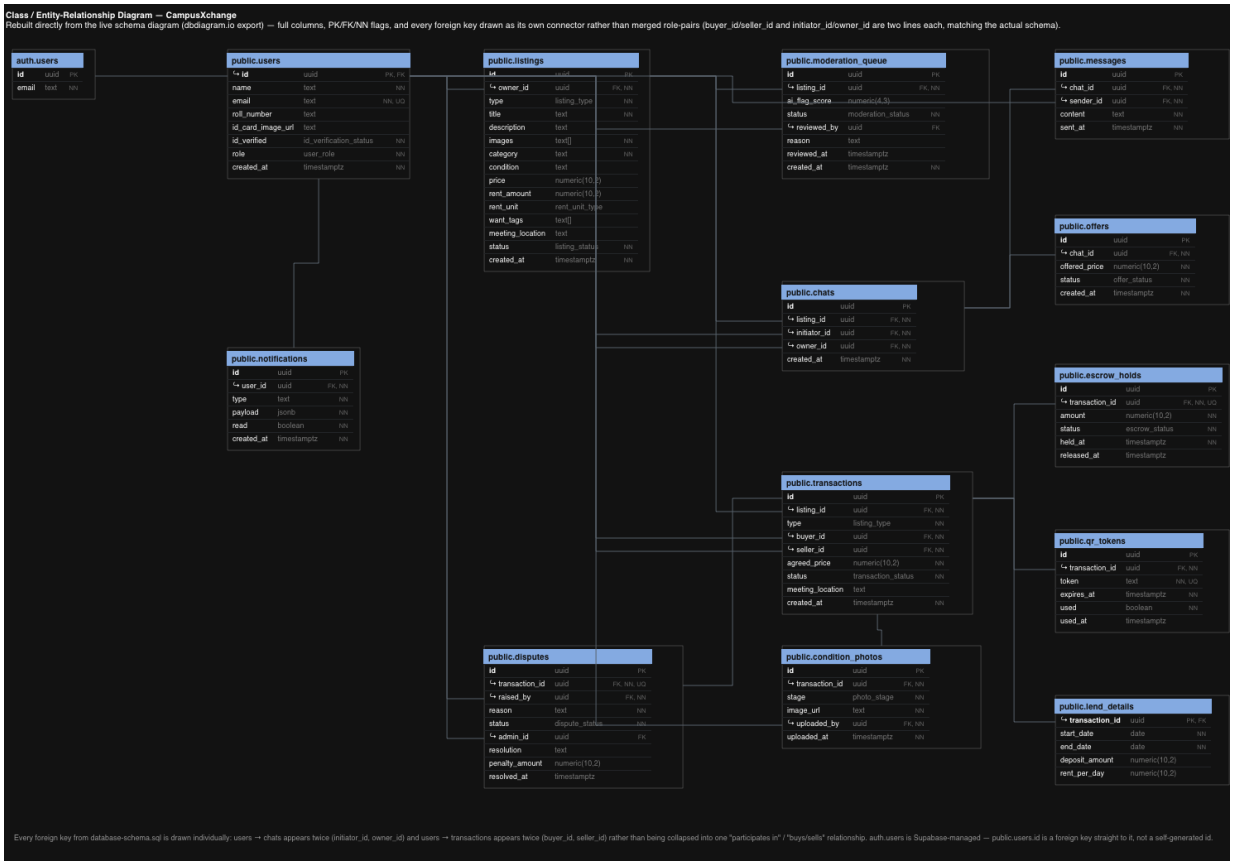


Figure 6.2.: Class / ER diagram (13 tables)

6.3. Data Dictionary

Table 6.1.: Database tables

Table	Purpose and key attributes
`users`	Profile extending Supabase `auth.users`: id (FK), name, email, roll_number, id_card_image_url, id_verified, role, created_at
`listings`	owner_id, type, title, description, images[], category, condition, price / rent_amount + rent_unit / want_tags[], meeting_location, status
`moderation_queue`	listing_id, ai_flag_score, status, reviewed_by, reason, reviewed_at
`chats`	listing_id, initiator_id, owner_id; unique (listing_id, initiator_id)
`messages`	chat_id, sender_id, content, sent_at
`offers`	chat_id, sender_id, offered_price, status
`transactions`	listing_id, type, buyer_id, seller_id, agreed_price, status, meeting_location
`escrow_holds`	transaction_id (unique), amount, status, held_at, released_at
`qr_tokens`	transaction_id, token (unique), expires_at, used, used_at
`lend_details`	transaction_id (PK), start_date, end_date, deposit_amount, rent_per_day
`condition_photos`	transaction_id, stage (handoff / return), image_url, uploaded_by

Table 6.1.: Database tables

Table	Purpose and key attributes
`disputes`	transaction_id (unique), raised_by, reason, status, admin_id, resolution, penalty_amount
`notifications`	user_id, type, payload (JSON), read

6.4. State Models

Table 6.2.: Lifecycle states

Entity	States and transitions
Listing	`pending_review` → `live` (approved) or `removed` (rejected / deleted); `live` → `reserved` (buyer selected) → `completed`
ID verification	`pending` → `verified` or `rejected`; `rejected` → `pending` on re-upload
Offer	`pending` → `accepted` or `rejected`
Transaction	`pending_payment` → `escrow_held` → `qr_issued` → `completed`; from `qr_issued` → `disputed` → `completed` or `refunded`
Escrow	`held` → `released` or `refunded`
Dispute	`open` → `under_review` → `resolved`

6.5. Sequence Diagrams

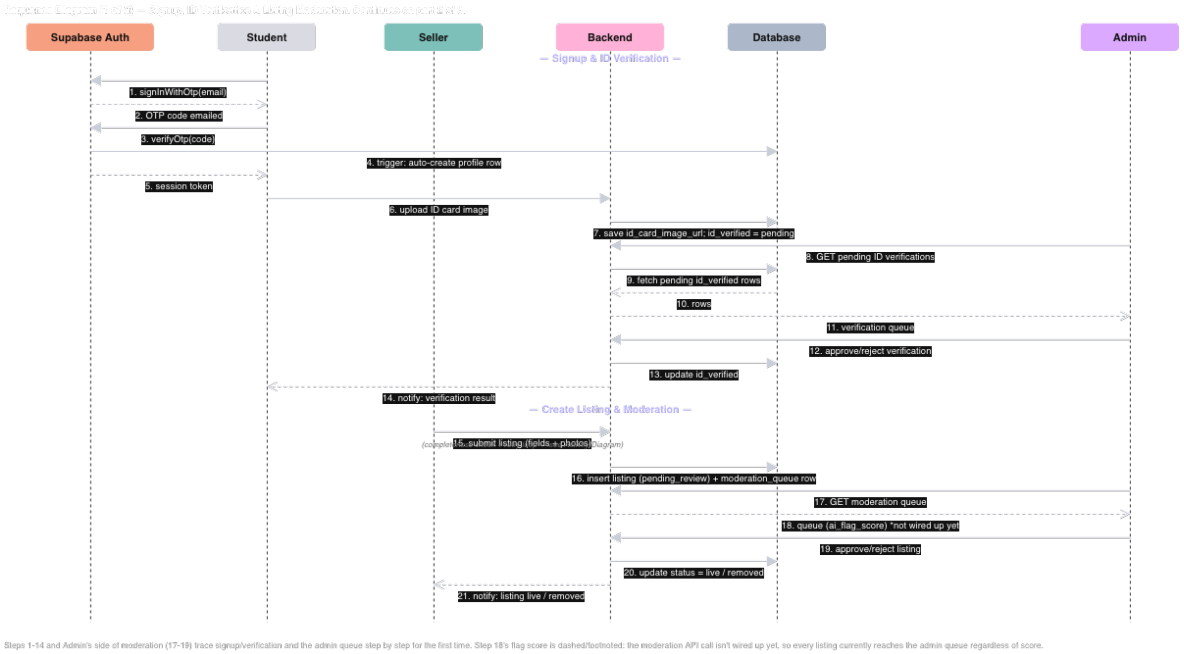
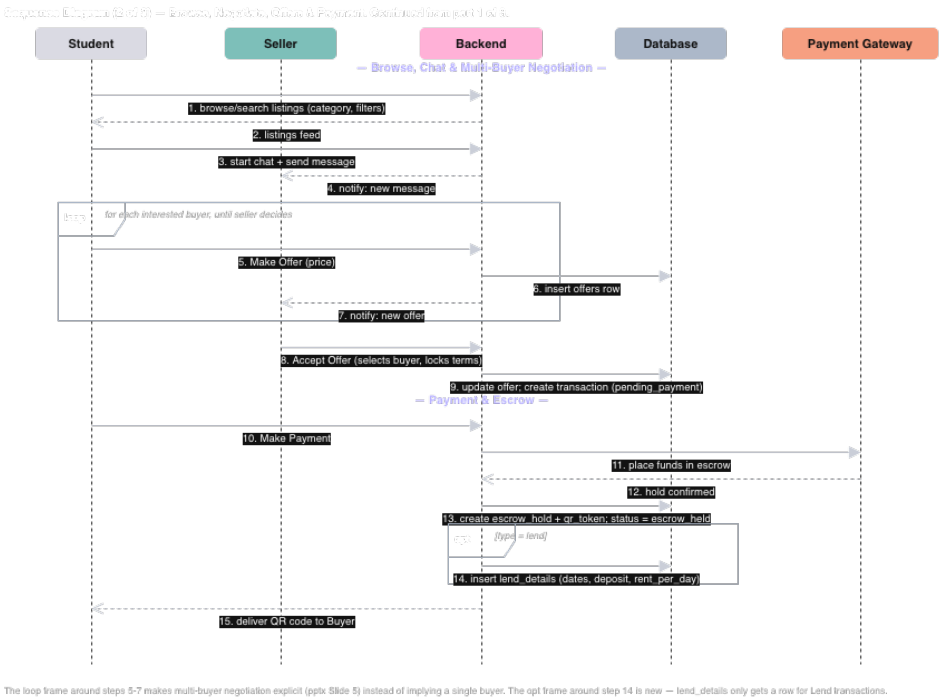
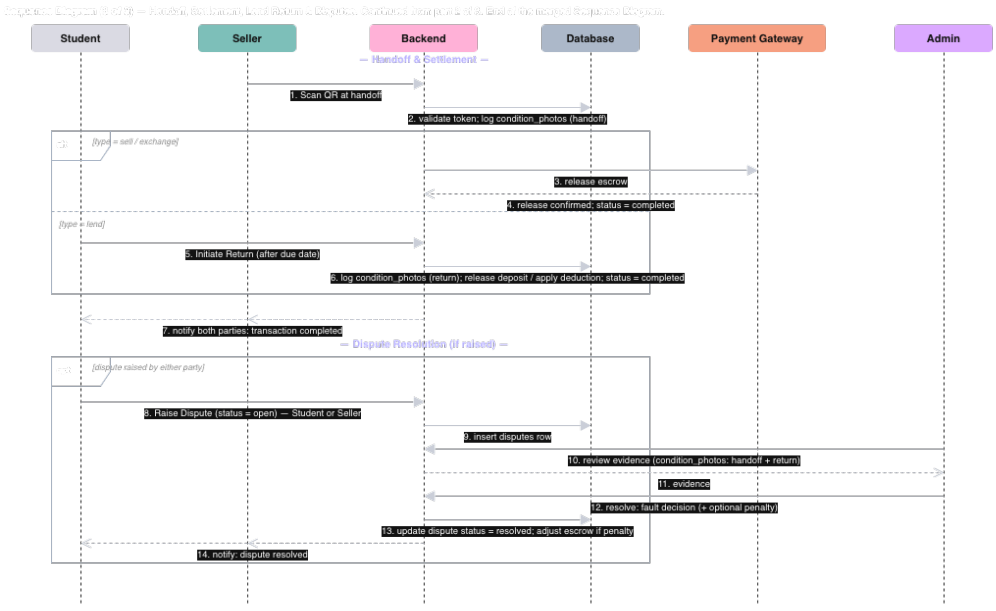


Figure 6.3.: Sequence diagram 1: sign-up, ID verification, listing and moderation



The loop frame around steps 5-7 makes multi-buyer negotiation explicit (pptx Slide 5) instead of implying a single buyer. The opt frame around step 14 is new — lend_details only gets a row for Lend transactions.

Figure 6.4.: Sequence diagram 2: browsing, multi-buyer negotiation, payment and escrow



The alt fragment (steps 3-6) is the part that's easy to skip — Lend's deposit/return cycle now has its own branch instead of silently reusing the Sell release path. Dispute steps 8-14 match the Activity Diagram's dispute flow one-for-one, at message level.

Figure 6.5.: Sequence diagram 3: QR handoff, lend return, settlement and disputes

6.6. Activity Diagrams

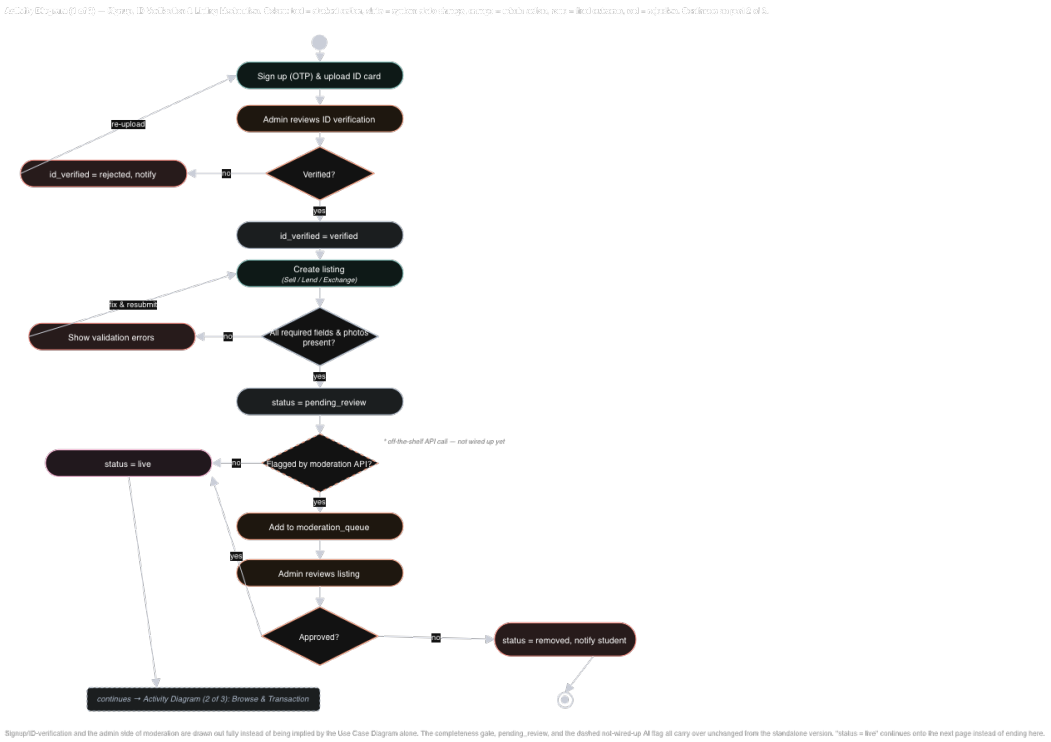


Figure 6.6.: Activity diagram 1: verification and moderation (with rejection loops)

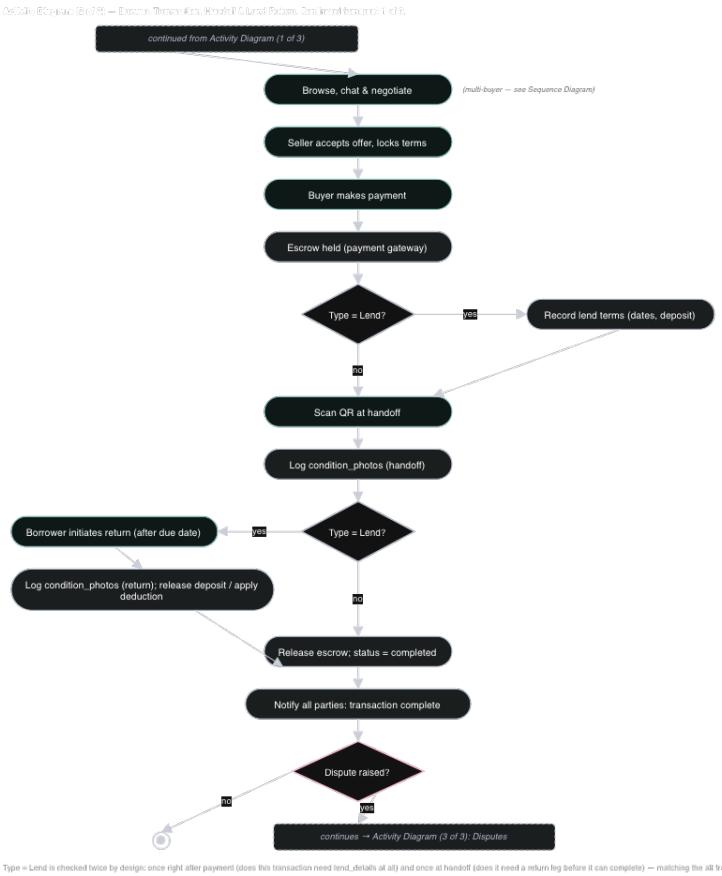


Figure 6.7.: Activity diagram 2: negotiation, payment, handoff and lend return

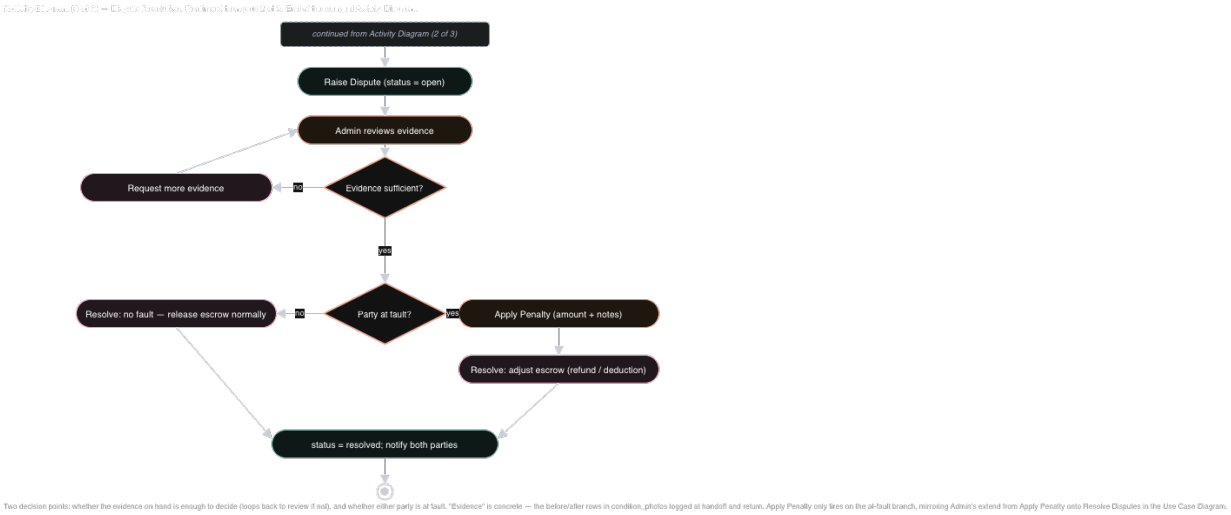


Figure 6.8.: Activity diagram 3: dispute resolution and penalties

7. Business Rules

BR-1. Only `@thapar.edu` email holders may register.

BR-2. Only verified students may create listings, negotiate, transact or raise disputes; guests may only browse.

BR-3. No listing is visible to others until an administrator approves it.

BR-4. A user cannot chat on, make offers on, or buy their own listing.

BR-5. Only the recipient of an offer may accept or reject it; one accepted offer closes negotiation in that thread.

BR-6. The seller chooses which buyer to finalise with when several are negotiating.

BR-7. Money is released to the seller only after a valid QR handoff (and return, for lend).

BR-8. Condition photographs are mandatory at handoff and at return.

BR-9. Each transaction can have at most one dispute; escrow is frozen while it is open.

BR-10. Automated tools may flag content but never reject, ban or penalise; an administrator makes every final decision.

BR-11. All meetings take place at approved on-campus meeting points.

8. Acceptance Criteria and Traceability

8.1. Evaluation Metrics

Table 8.1.: Pilot metrics (from the proposal)

Metric	Definition	Target
Time to listing clearance (primary)	Submission to moderation decision, from database timestamps	Median $\leq$ 4 hours
Transaction completion rate	Agreed transactions reaching QR-verified completion	Tracked; increasing across iterations
Lend share and on-time return	Share of lend deals; returns by due date without dispute	Tracked
Dispute rate	Lend deals flagged after photo comparison	Tracked; decreasing
Off-platform discovery	Pilot users finding/listing in-app vs. WhatsApp	Majority in-app
User clarity	1–5 rating of transaction-status clarity	≥ 4
Reliability	Uptime of login, listing, payment at peak	$\geq 99\%$

The pilot involves 30–40 students (freshers, graduating seniors and hostel residents) and 2–3 volunteer moderators.

8.2. Traceability Matrix

Table 8.2.: Objectives and proposal features traced to requirements

Objective / pain point	Proposal feature	Requirements
O1 / "Strangers, no proof"	Verified access	FR-AUTH-1–10, FR-ADM-3–4, NFR-SEC-1–4
O2 / "No single place to look"	Sell, Lend, Exchange modules	FR-BRW-1–8, FR-LST-1–9
O3	Trust & safety layer (moderation)	FR-MOD-1–7, FR-ADM-1–2
O4	In-app chat & negotiation	FR-CHT-1–10
O5 / "No recourse"	Payments (escrow), QR handoff	FR-PAY-1–6, FR-HND-1–5
O5 / "Lending has no memory"	Lend/Borrow module	FR-LND-1–4, FR-PAY-3

Table 8.2.: Objectives and proposal features traced to requirements

Objective / pain point	**Proposal feature**	**Requirements**
O6	Admin dispute handling	FR-DSP-1–6
O7	Evaluation (TLC)	FR-MOD-5, FR-ADM-5

A. Known Gaps and Open Issues

These are acknowledged differences between this specification and the current build, to be resolved before the pilot.

1. **Verification gate not enforced on actions.** The backend lets any signed-in student create listings and chat; FR-LST-1 and BR-2 require `id_verified = verified`. A guard on these endpoints is needed.
2. **Row-Level Security disabled** on all 13 tables (NFR-SEC-5).
3. **Chat uses polling** (4 s) instead of WebSockets (FR-CHT-10).
4. **Backend not yet deployed**; testing is limited to a shared local network (NFR-SCL-2).
5. **Payment gateway not yet selected**; the escrow model assumes a gateway with hold/release or split-payment support.
6. **Penalty tiers** for disputes must be defined before the dispute module is built (proposal risk mitigation).